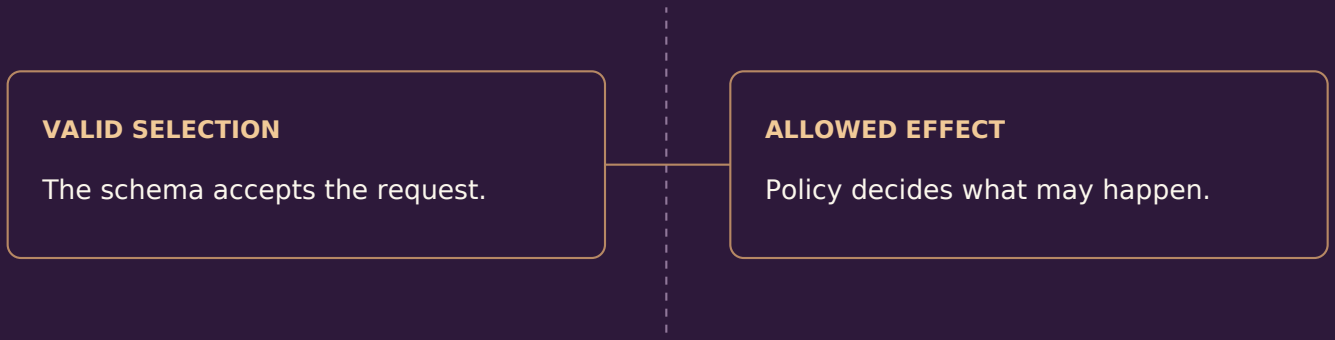
SCHEMA / EXECUTION / ACCESS CONTROL

GraphQL API Security Explained

Understand execution, prove access failures, and build a safer graph



Understand the graph before testing it.
Follow the resolver. Prove the effect. Verify the repair.
Practice with a real engine and fictional project data.

Haroon Shahid

First edition / Version 1.0 / 4 November 2026

Before you begin

GraphQL API Security Explained

Understand execution, prove access failures, and build a safer graph

By Haroon Shahid. First edition, version 1.0. 4 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code has its own license. Prepared with AI assistance. For corrections: gaknt2@gmail.com. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework and protocol guidance is version-sensitive. The lab runs a real GraphQL engine with local HTTP and WebSockets, fixed dummy identities, and synthetic records. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package adds the complete curriculum, answered reviews, workbook, reference sheets and Kestrel Graph Studio.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. What an API does 6

02. Read HTTP without guessing 8

03. REST and GraphQL solve different problems 10

Appendix A. Answer explanations 12

Sources and further reading 13

Introduction

Maya can open Harbor. She cannot open Hill. Both projects have the same GraphQL type. The schema accepts the same kind of ID for both. Why must one request return data while the other returns nothing? That gap between a legal request and an authorized result is where this book begins.

Kestrel Projects is our fictional app. You will follow the same actors and records through API basics, real GraphQL execution, object and field access, writes, demand control, caches, subscriptions and service handoffs. Each chapter offers a small prediction. Try answering it before reading the explanation.

The working model is **request → schema → resolver → policy → effect**. Ask what the caller chose, what the engine accepted, which path reached the data, whose authority was used, and what you observed. Apply it to every path, including ones the main screen never calls.

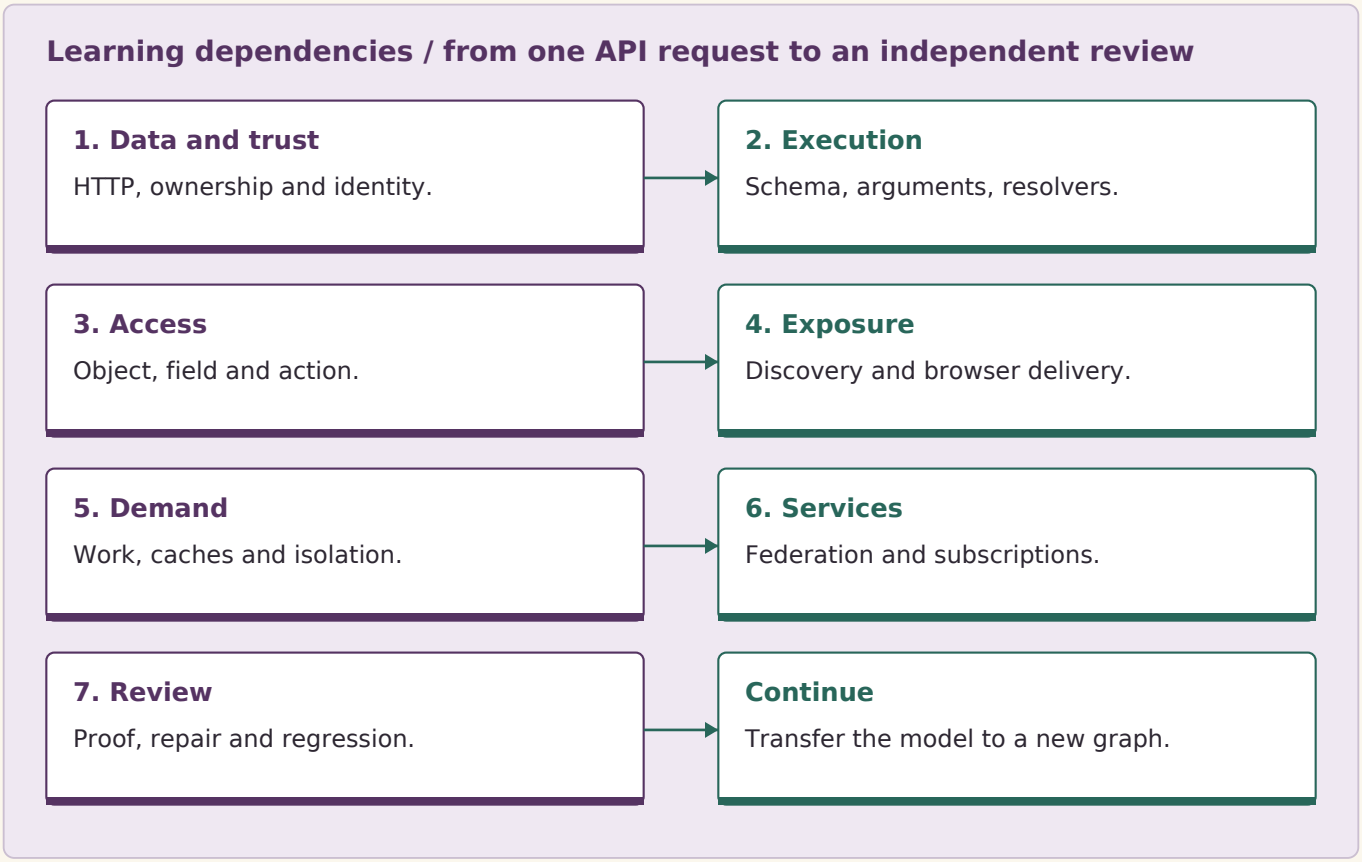
You need no GraphQL experience to start. If you already build APIs, use the execution walkthrough in Chapter 11 as a placement check. If you test APIs, start at Chapter 13 and come back whenever validation, nulls, pagination or identity changes your interpretation.

The local Kestrel Graph Studio uses an actual GraphQL.js engine, loopback HTTP and WebSockets, and fictional in-memory data. Some routes are intentionally weak. Compare them with repaired routes and positive controls. The architecture cases are separate from the features executed in the lab. See the included validation report for that boundary.

Learning roadmap

Stage	Show that you can
1. API and trust	Trace a request and write a permission sentence.
2. Graph execution	Predict field execution and explain partial data.
3. Access and actions	Prove an unauthorized read or state change with two accounts.
4. Exposure and transport	Distinguish discovery, browser delivery and actual impact.
5. Resource and cache limits	Bound fan-out and preserve data isolation.
6. Distributed graph	Recheck identity, service trust and event delivery.
7. Independent review	Reproduce, repair, retest and report a bounded claim.

Read the four nodes of each figure in their labeled order. Some figures show alternatives rather than a timeline. After each part, close the book and reconstruct one decision from memory.



Read each row left to right, then move down. Each later stage uses the earlier mental model.

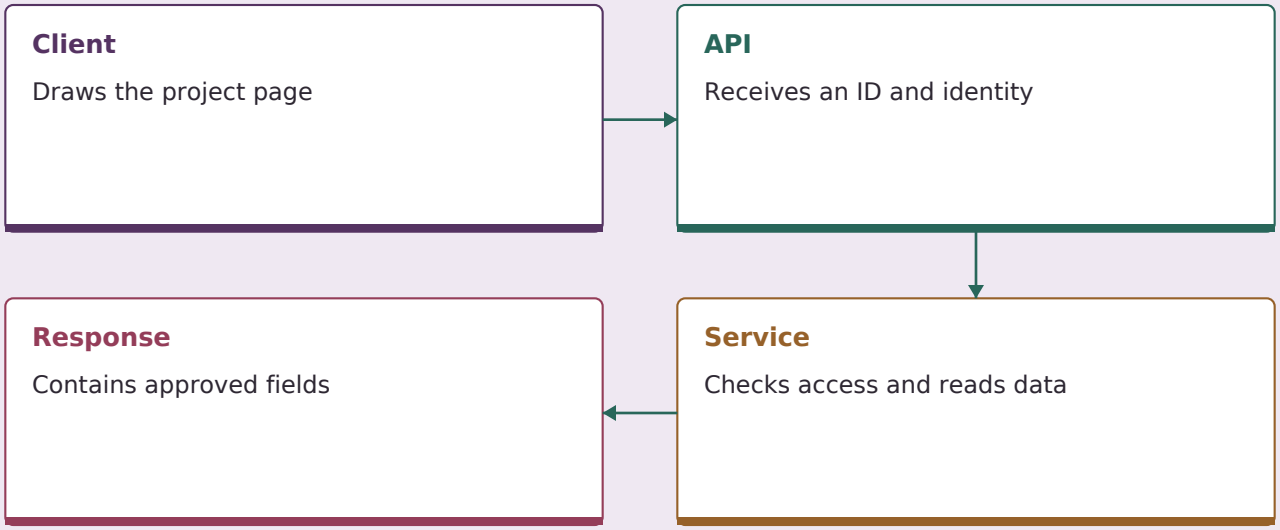
I. UNDERSTAND DATA, APIS AND TRUST

01. What an API does

Learn to: Trace one request from a screen to stored data and back.

Think of it this way: A library desk accepts a book number, checks the membership card and returns what the member can borrow.

01A / One screen, several decisions



The app can request a project, but the server decides what to retrieve and reveal.

Follow one project

Kestrel Projects is a fictional place where teams track work. Pine and Vale are two separate organizations. A project called Harbor belongs to Pine. A project called Hill belongs to Vale. If Maya signs in to Pine, the screen should show Harbor. To get the data, the screen sends a request to an API.

An **API** is an agreed way for software to ask other software to do something. The browser sends a message. The server understands the message, calls the right code and replies. That code may read a database, ask another service, or decide that the request is not allowed.

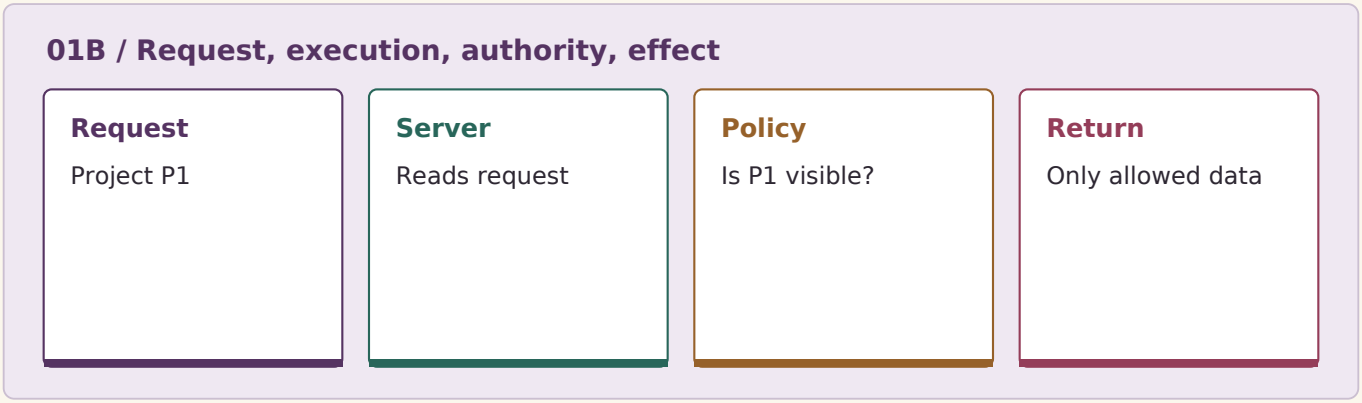
The screen and API are different layers. The screen can hide a button or only show one project's link. Someone who is allowed to test the service can still send an API request directly. If the API accepts a project ID, it must check permission for the requested project even when the usual screen would never send that ID.

Data, permission and output

A database stores facts. A service decides which facts may be used for a given request. A response is the result of that decision. Treating these steps as one hides useful questions. Did the service get the wrong record? Did it fetch the right record but show a private field? Did it return the right data to the wrong user through a cache?

Practice

Draw the browser, API, service and data store for an assignment page. Label one input the user can change and one fact the server must obtain independently. Use Maya's membership as the server-owned fact.



Key takeaway. A visible screen is only one caller. The API protects data for every caller.

Check your understanding

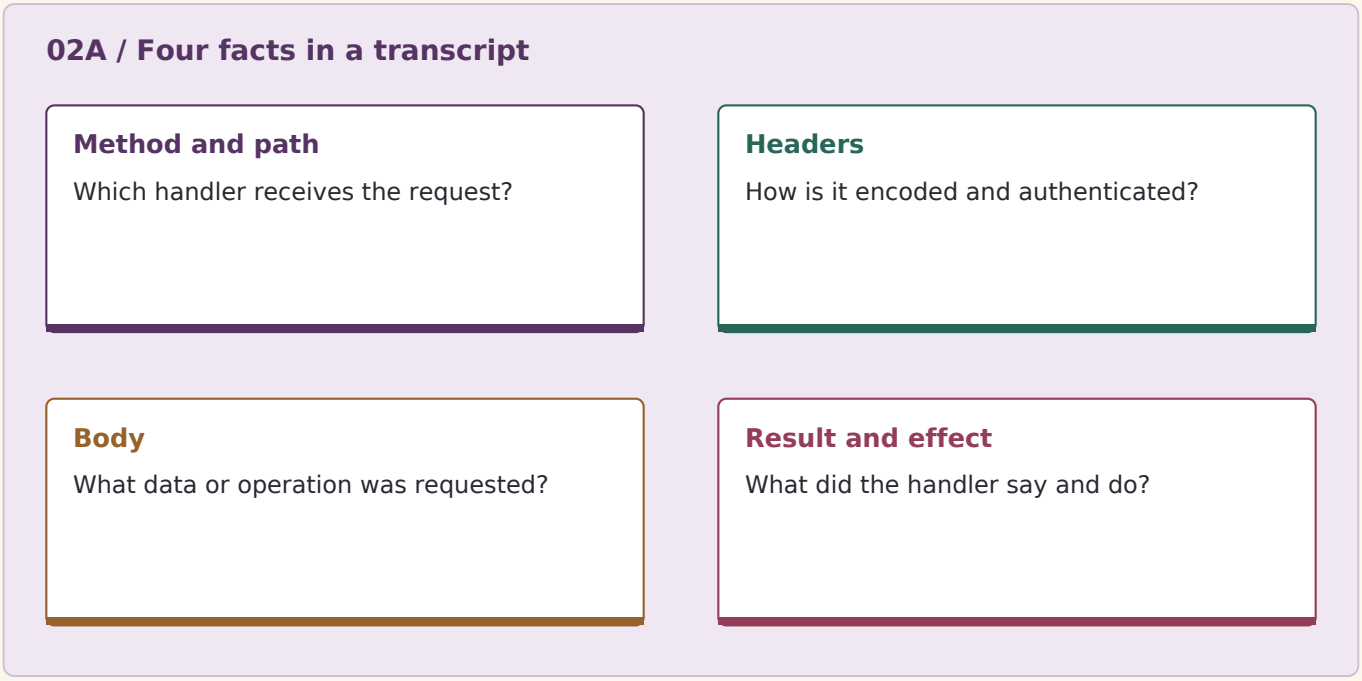
1.1 Does hiding a button stop API calls?	**1.2** What makes Pine and Vale separate tenants?	**1.3** Which part decides what to return?

Answer explanations: Appendix A. Sources: S01, S02, S03

02. Read HTTP without guessing

Learn to: Separate a transport result, an application decision, and a change in stored state.

Think of it this way: A courier can deliver an envelope even if the person opening it rejects the request inside.



A transport status needs the message body and actual effect for interpretation.

The message has parts

HTTP carries requests and responses. A request has a method, a target and headers. It can have a body. A response has a status, headers and usually a body. HTTPS protects the transport with TLS when correctly configured. It does not decide whether Maya should see a Vale project.

Headers can describe how to interpret a body or carry a credential. The Content-Type header can say the body is JSON. The Accept header asks for a response representation. A bearer token or cookie can help the server establish identity. A user-supplied header called X-Tenant, by itself, is just a claim.

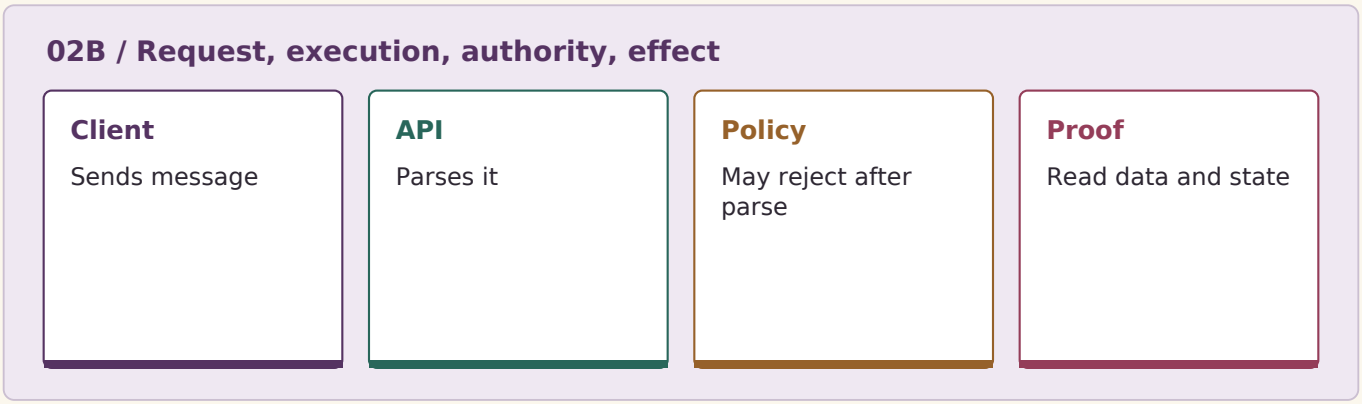
The status code is a clue

A service can accept a well-formed request, fail one operation and still return a response containing useful details. With GraphQL, a response can contain both data and errors. A status of 200 does not show that every field succeeded. A null value does not tell you, by itself, whether a field is absent, denied, or affected by an error.

For a mutation, check what persisted. An error returned after a write does not undo the write. Compare a baseline and one permitted synthetic action. Inspect the response body and then read the resulting state using the allowed account.

Practice

In a training transcript, circle the URL, method, media type, identity carrier, requested object and evidence of effect. Mark any conclusion the transcript does not establish.



Key takeaway. Read transport, GraphQL result and business effect as separate observations.

Check your understanding

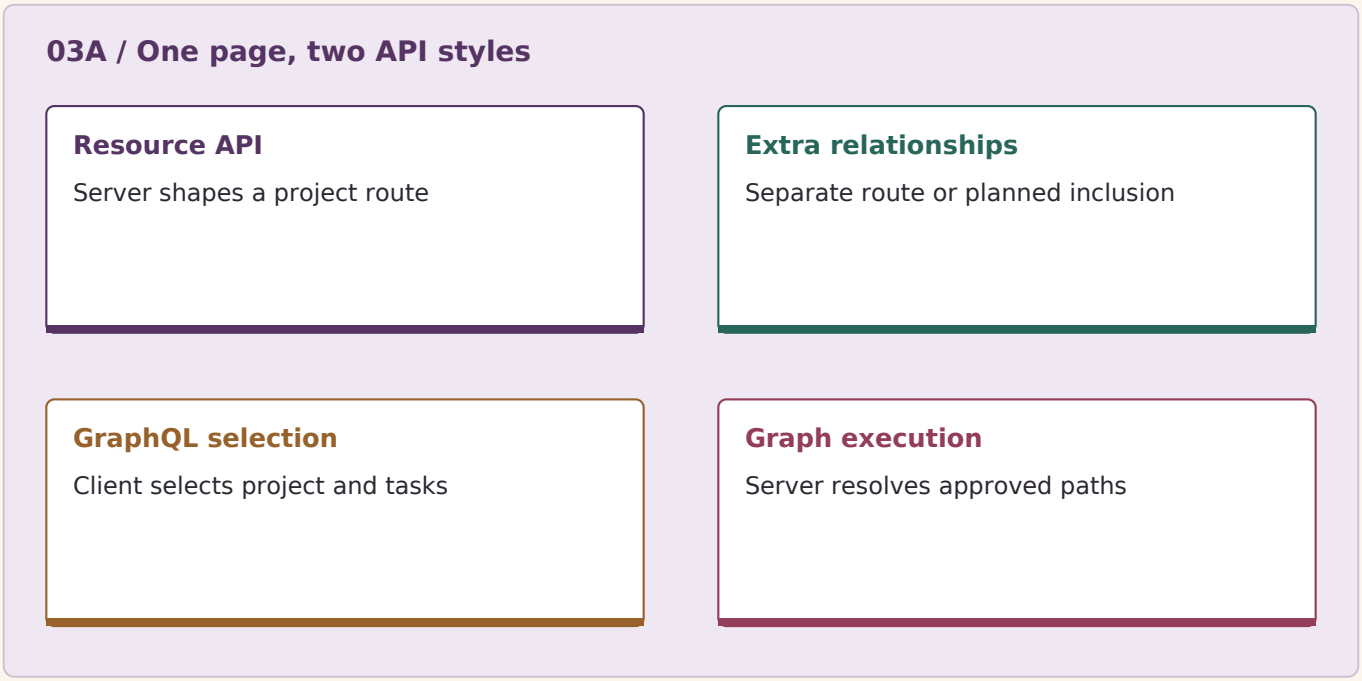
2.1 Does HTTPS decide object access?	**2.2** Is a tenant header proof of membership?	**2.3** What evidence is strongest for a write?

Answer explanations: Appendix A. Sources: S01, S04, S05

03. REST and GraphQL solve different problems

Learn to: Explain why a product chooses a queryable graph and where the cost moves.

Think of it this way: A set menu gives you fixed combinations. A menu with choices gives you flexibility and makes the kitchen coordinate more orders.



Both styles can serve the same product. Each needs access rules and workload limits.

Start with the user interface

The project page needs a title, the first three tasks and each task's status. A resource-based HTTP API could return a fixed project representation and provide a separate task endpoint. It could also accept a carefully designed include option. REST is an architectural style; it is not a rule that every screen must use many calls.

A GraphQL API describes a typed graph. A client writes a selection of fields it wants, and the server produces a matching response shape. The server can evolve the schema with new fields while older clients continue selecting the fields they know. GraphQL does not replace the database. Its resolvers can call services, databases, caches or other APIs.

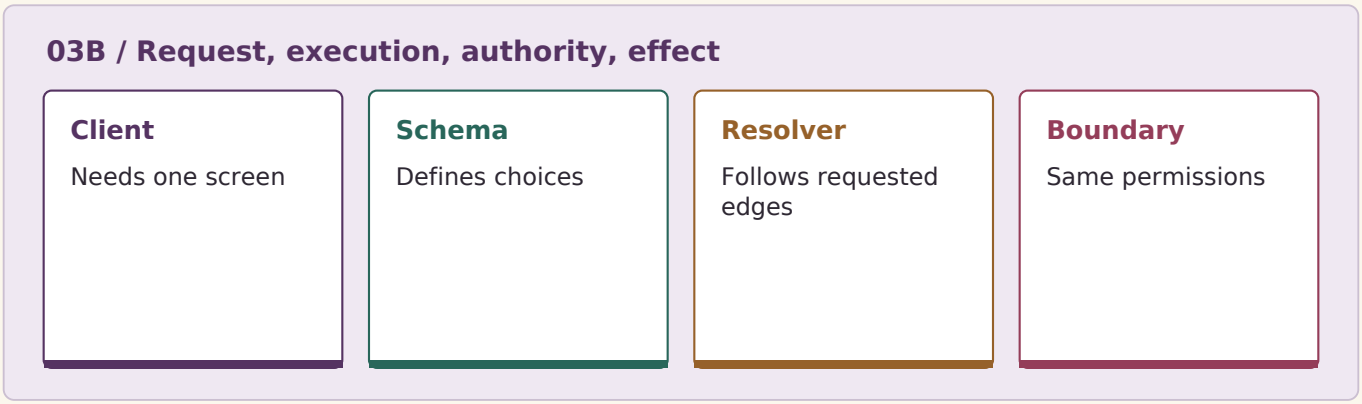
What flexibility changes

A client can ask for tasks under project, or projects under organization if the schema permits it. A designer must choose how deep the available relationships go, how lists are bounded, and where identity and policy travel. A flexible contract can reduce fixed endpoint combinations, but it creates more paths to the same data.

GraphQL does not make REST obsolete, and either style can have broken access control. Pick the API style for product needs, then review the actual paths. Count resolver calls and downstream work instead of equating one HTTP request with one database read.

Practice

Design two legal requests for the same screen: one resource API call sequence and one GraphQL selection. Mark the object and field permissions each implementation needs.



Key takeaway. The client can shape its request. The server still owns permission and the cost budget.

Check your understanding

3.1 Is GraphQL a database?	**3.2** Can REST return related records in one response?	**3.3** What security concern grows with selectable edges?

Answer explanations: Appendix A. Sources: S02, S03, S06

Appendix A. Answer explanations

Chapter 01: What an API does

1.1 Does hiding a button stop API calls? No. Server checks must apply even if someone skips the usual interface.

1.2 What makes Pine and Vale separate tenants? Each has its own members and project boundary.

1.3 Which part decides what to return? Server-side application policy, backed where possible by scoped data access.

Chapter 02: Read HTTP without guessing

2.1 Does HTTPS decide object access? No. It protects transport, while policy decides access.

2.2 Is a tenant header proof of membership? No. Verify it against trusted identity and current membership.

2.3 What evidence is strongest for a write? A controlled before-and-after observation of the intended record.

Chapter 03: REST and GraphQL solve different problems

3.1 Is GraphQL a database? No. It defines a contract and execution model that can call many data sources.

3.2 Can REST return related records in one response? Yes. The API can design explicit representations and inclusion.

3.3 What security concern grows with selectable edges? Several paths may reach the same protected data or trigger more work.

Sources and further reading

Research cutoff: 4 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

S01 - RFC 9110: HTTP Semantics

Methods, response semantics and representations. Use alongside the GraphQL transport draft.

https://www.rfc-editor.org/rfc/rfc9110.html

S02 - GraphQL: Learn

Accessible overview of graphs, types and client-selected fields.

https://graphql.org/learn/

S03 - GraphQL: Authorization

Authorization as business logic; placement of reusable policy behind resolvers.

https://graphql.org/learn/authorization/

S04 - GraphQL over HTTP draft

Draft transport contract for method, media type and status. Confirm the deployed server's behavior.

https://http-spec.graphql.org/draft/

S05 - GraphQL: Serving over HTTP

HTTP envelope, media negotiation and response data/errors.

https://graphql.org/learn/serving-over-http/

S06 - Fielding: REST architectural style

Original context for REST constraints; REST is not simply JSON routes.

https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm